

K.R. Mangalam University

DSA Assignment:2

Submitted by:

Madhav kalra

2501730284

B.TECH CSE(AI&ML)

Section: A

Submitted to:

Mrs. Neetu Chauhan

Task 1: Dynamic Array Simulation:

1. Dynamic Array – Amortized Time Complexity:

In Dynamic Array, most of the time when we use `append()`, the element is added at the end directly, so it takes **$O(1)$** time but in case the array is full we need to create a new array of double size and copy all elements to create a new one. However, resizing does not happen every time. It happens only occasionally. So if we consider many operations together, the average time per operation becomes **$O(1)$** this is called **amortized time complexity**. But when we do this complexity on copying it takes **$O(n)$** complexity.

2. Linked List Operations:

Linked lists are useful because they allow dynamic memory allocation. Unlike arrays, they do not require contiguous memory.

3. Stack & Queue From Scratch (Using Singly Linked List):

✧ Stack Operations

- **`push(x)`** $\rightarrow O(1)$
Element is inserted at the top (head), so it takes constant time.
- **`pop()`** $\rightarrow O(1)$
Top element is removed directly, no traversal needed.

- **peek()** $\rightarrow O(1)$
Only returns the top element without removing it.

✧ Queue Operations

- **enqueue(x)** $\rightarrow O(1)$
Element is added at the rear (tail) using a pointer.
- **dequeue()** $\rightarrow O(1)$
Element is removed from the front (head) directly.
- **front()** $\rightarrow O(1)$
Returns the front element without removing it.

4. Parentheses Checker (Using Your Stack) :

Opening brackets are pushed into the stack, and when a closing bracket appears, the top element is checked and removed. If all brackets are matched correctly and the stack becomes empty at the end, the expression is balanced.

Conclusion:

Data Structure	Operation	Time Complexity
Stack	Push	$O(1)$
Stack	Pop	$O(1)$
Stack	Peek	$O(1)$
Queue	Enqueue	$O(1)$
Queue	Dequeue	$O(1)$
Queue	Front	$O(1)$